

FCN - Indice rapido delle funzioni esportate

Alias, costanti ed enumerazioni

- `using Vec = std::vector<double>;`
- `using VecN = std::vector<int>;`
- `using Mat = std::vector<Vec>;`
- `using KM = matplot::keyword_manual_type;`
- `using KA = matplot::keyword_automatic_type;`
- `const double PI = std::acos(-1.0);`
- `enum class SortOrder { Asc, Desc };`

funzioni di gestione interfaccia utente

- `void cin_clear();`
- `string color_bool(const bool val);`
- `string color_dbl(const double val);`
- `void color_rst();`

funzioni di conversione formato ed equivalenza unita' di misura

- `std::string itostr(const int nn);`
- `double deg2rad(const int alpha);`
- `string format_numstr(double v);`

funzioni di base per intervalli

- `double h_ticks(const double a_start, const double a_stop, const int a_points);`
- `Vec nodi_bubblesort(const Vec x_uns, const int totnum);`
- `Vec nodi_equidistanti(const double amin, const double amax, const int NPoints);`
- `Vec nodi_random(const double amin, const double amax, const int NPoints);`

funzioni matematiche ad uso callback

- `double f_x(double t);`
- `double f_sin(double t);`
- `double f_cos(double t);`
- `double f_atan(double t);`
- `double f_atan_d(double t);`
- `double f_sin2plus1(double t);`
- `double fcallb(double t, double (*f)(double));`

funzioni macro algebra lineare

- `Vec linear_subst_BW(const Mat &U, const Vec &y);`
- `Vec linear_subst_FW(const Mat &L, const Vec &b);`
- `void linear_jacobi_autoval_simmetrica(const Mat& A, Vec& lambda, Mat& V);`

- `Vec linear_LU_calcola_autovalori(const Mat& A);`
- `void linear_LU_dec(const Mat &A, Mat &L, Mat &U);`
- `Mat linear_LU_inversa(const Mat& A);`
- `Vec linear_LU_risolve_colonna(const Mat& L, const Mat& U, Vec x);`
- `Vec linear_LU_risolve_sistema(const Mat& T, const Vec& x);`
- `double linear_max_autoval_pwr_any_res(const Mat& M, int max_iter, double tol);`
- `double linear_max_autoval_pwr_any(const Mat& M, int max_iter, double tol);`
- `double linear_max_autoval_pwr_AtA(const Mat& M, int max_iter, double tol);`

funzioni per la gestione di vettori e matrici

- `Mat matrix_build_derivata1(int n, double a, double b);`
- `Mat matrix_build_gausskernel(const int N, const double sigma, const double h, bool norm_flag, Vec& Indicatori);`
- `Mat matrix_build_gram(const Vec& x, const int K);`
- `Mat matrix_build_Id(int n);`
- `Mat matrix_build_triang(int n);`
- `Mat matrix_build_triang_inv(int n);`
- `Mat matrix_build_zero(int righe, int colonne);`
- `double matrix_calcola_errone_Fr(const Mat& A, const Mat& B);`
- `void matrix_calcola_media(const Mat& A, Vec& avg_col, Vec& avg_row);`
- `double matrix_calcola_norma(int norma, const Mat& A);`
- `Mat matrix_centra_su_media(const Mat& A, const Vec& avg_vec, bool by_col = true);`
- `Mat matrix_differenza_dump(const Mat& A, const Mat& B);`
- `void matrix_dump(const Mat &A, const std::string &nome);`
- `Mat matrix_estende_ridotta(const Mat& A, int n, bool bycol);`
- `Mat matrix_normalize_byrow(Mat& K);`
- `void matrix_ordina_diagonale(Vec& lambda, Mat& V, double zero_tol = 0.0, SortOrder order = SortOrder::Desc);`
- `void matrix_ortogonalizza_GSmod(Mat& Q, int j0 = 0);`
- `Mat matrix_prodotto_coeff(const Mat& A, const double coeff);`
- `Mat matrix_prodotto_AtA(const Mat& A, bool A_right = true);`
- `Mat matrix_prodotto_matrix(const Mat& A, const Mat& B);`
- `Vec matrix_prodotto_vector(const Mat& A, const Vec& v);`
- `void matrix_test_ortogonale(const Mat& A, string s);`
- `Mat matrix_trasposta(const Mat& A);`
- `Vec vector_add_noise(Vec& v, double e);`
- `Vec vector_build_versore_canonico(int j, int N);`
- `double vector_calcola_norma(int norma, const Vec& V);`
- `Vec vector_campiona_f(const Vec &x, double (*ft)(double));`
- `Vec vector_campiona_f_k(int k, const Vec &x, double (*ft)(double));`
- `void vector_dump(Vec x, int colspan, int totnum, const std::string s);`
- `Vec vector_householder_bycol(const Vec& v);`
- `Vec vector_householder_byrow(const Vec& v);`
- `double vector_prodotto_scalare(const Vec &u, const Vec &v);`
- `Vec vector_reverse(const Vec& v);`
- `Vec vector_segnalet_finestra(int N, double a, double b, double t);`

- `Vec vector_shift(const Vec &v, const double shift);`
- `Mat vector_to_matrix(const Vec& v, bool transp);`
- `Mat vector_to_matrix_diag(const Vec& s, int m, int n);`
- `void trmatrix_bidiagonalizza(const Mat& A, Mat& U0, Mat& B, Mat& V0, bool sup_diag = false, bool dump_flag = false);`
- `void trmatrix_bidiag_wide_to_lower(const Mat& X, Mat& U0, Mat& B, Mat& V0, bool dump_flag);`
- `void trmatrix_bidiag_wide_to_upper(const Mat& X, Mat& U0, Mat& B, Mat& V0, bool dump_flag);`
- `void trmatrix_SVDQR(const Mat& B, Mat& Ub, Mat& Vb, Vec& sigma);`
- `void trmatrix_SVDQR_ridotta(const Mat& B, Mat& Ub, Vec& sigma, Mat& Vb_red, double ev_tol = 1e-12);`
- `void trmatrix_test_sv_autoval(Vec lambda, const Vec& sigma);`

Funzioni per matplotlib++

- `void matplotlib_legend_align(legend_handle lg, int pos_enum, float xscale, float yscale);`
- `figure_handle matplotlib_table_init(const bool ahold, const std::string &nome, const std::string &titolo, const int xlab, const int ylab);`

gestione menu principale

- `struct MenuItem { int key; std::string label; std::string action; bool enabled; };`
- `struct MenuConfig { std::string title; std::vector<MenuItem> items; };`
- `MenuConfig load_menu_config(const std::string& filename);`
- `const MenuItem* find_menu_item(const MenuConfig& menu, int key);`
- `void wait_return_to_menu(bool bypass_waitakey);`